PROJETO INTEGRADOR FULL STACK — 1ª ENTREGA (FASES 1 A 4)
MODERNIZAÇÃO DE SISTEMA LEGADO: COOPERATIVAS DE CRÉDITO

Mateus Henrique Szabo | Tema Base Legada: 17_cooperativas_credito | Stack Alvo: Java 21, Spring Boot, JPA/Hibernate, Spring Security, Thymeleaf, Bootstrap e H2

1. CONTEXTUALIZAÇÃO DO DOMÍNIO E ESCOPO DA MODERNIZAÇÃO

Este documento formaliza os artefatos referentes à 1ª Entrega (Fases 1 a 4) do Projeto Integrador Individual de Programação Web. O objeto de estudo consiste na base de dados legada de um ecossistema de Cooperativas de Crédito (arquivo 17_cooperativas_credito), cujo período de cobertura compreende operações financeiras e indicadores registrados entre janeiro de 2024 e dezembro de 2025. O domínio abrange 10 cooperativas de crédito distribuídas nacionalmente, 6.000 associados (pessoas físicas, produtores rurais, profissionais liberais, micro e pequenas empresas e cooperados institucionais), 7.200 contas bancárias, 5.000 contratos de operações de crédito, 16.000 transações financeiras, 6.240 apontamentos mensais de inadimplência, 4.200 aplicações de investimento e 240 registros de indicadores gerenciais mensais.

A análise crítica da estrutura original demonstrou que o sistema legado operava como um repositório desnormalizado e desprovido de validação de regras de negócio no servidor. O propósito desta modernização em arquitetura Full Stack (Java 21, Spring Boot 3, Spring Data JPA/Hibernate, Spring Security, Thymeleaf e Bootstrap) não se limita à migração de tabelas, mas estabelece integridade referencial estrita em 3ª Forma Normal (3FN), máquina de estados determinística para contratos e transações, cálculos financeiros automatizados com precisão decimal (BigDecimal) e controle de acesso baseado em perfis (RBAC) validado em camada de serviço.

2. DIAGNÓSTICO CRÍTICO DA BASE DE DADOS LEGADA (FASE 2)

A inspeção completa das 8 tabelas de dados da planilha original permitiu mapear a estrutura física legada e quantificar anomalias graves de integridade. O modelo original apoia-se em chaves numéricas sequenciais sem restrições de chave estrangeira compostas e replica o identificador da cooperativa (id_cooperativa) e do associado (id_associado) em tabelas de nível inferior sem garantir consistência com o registro pai.

2.1. Inventário das Tabelas, Chaves e Volume da Base Legada

Tabela Legada	Registros	Chave	Chaves Estrangeiras	Papel no Domínio e Problemas

		Primária (PK)	(FKs Legadas)	Estruturais
Cooperativas	10	id_cooperativa	Nenhuma	Cadastro matriz das cooperativas. Possui coluna estática associados_ativos que diverge da contagem real de associados.
Associados	6.000	id_associado	id_cooperativa	Cadastro de cooperados anonimizados. Usa strings 'Sim'/'Não' para flags lógicas e mantém coluna redundante possui_conta_digital.
Contas	7.200	id_conta	id_associado, id_cooperativa	Contas correntes, digitais, poupança e salário. A FK id_cooperativa é transitiva e diverge da cooperativa do associado em 90,3% dos casos.
Operacoes_Credito	5.000	id_operacao	id_associado, id_cooperativa	Contratos de empréstimo e financiamento. Possui FK id_cooperativa redundante e permite contratos ativos para associados inativos.
Transacoes	16.000	id_transacao	id_conta, id_associado, id_cooperativa	Movimentações (Pix, TED, Boleto, Débito, etc.). Dupla redundância transitiva (id_associado e id_cooperativa) e datas anteriores à abertura da conta.
Inadimplencia	6.240	id_registro	id_operacao, id_associado, id_cooperativa	Histórico mensal de atraso e provisão (PCLD). Carece de constraint UNIQUE por (id_operacao, mes_referencia) e aponta atrasos em contratos quitados.
Investimentos	4.200	id_investimento	id_associado, id_cooperativa	Aplicações em RDC, LCA, Cota Capital e Fundos. FK id_cooperativa redundante e status controlado por texto livre sem máquina de estados.
Indicadores_Mensais	240	Ausente (Composta implícita)	id_cooperativa	Painel mensal de desempenho por cooperativa. Não possui coluna de PK explícita nem constraint formal de unicidade mensal.

2.2. Auditoria Quantitativa de Anomalias e Inconsistências nos Dados Legados

Para comprovar empiricamente as fragilidades do sistema legado, realizou-se o cruzamento relacional de todos os registros da base. A tabela a seguir consolida as 17 falhas sistêmicas verificadas e a quantidade exata de registros corrompidos:

Tabela Afetada	Categoria da Falha	Descrição Técnica da Regra Violada	Registros Afetados	Proporção
Contas	Violação de 3FN / FK	O campo id_cooperativa da conta é diferente do id_cooperativa do seu titular (Associado).	6.503 de 7.200	90,32%
Contas	Status Inconsistente	Contas marcadas como ativas ('Sim') cujo titular encontra-se inativo ('Não') na tabela Associados.	1.336 de 7.200	18,56%
Operacoes_Credito	Violação de 3FN / FK	O campo id_cooperativa do contrato de crédito diverge da cooperativa de origem do associado.	4.482 de 5.000	89,64%
Operacoes_Credito	Status Inconsistente	Contratos em status 'Ativo', 'Em atraso' ou 'Renegociado' vinculados a associados inativos.	754 de 5.000	15,08%
Transacoes	Violação de 3FN / FK	O id_associado gravado na transação não corresponde ao titular real da conta (id_conta).	15.996 de 16.000	99,98%
Transacoes	Violação de 3FN / FK	O id_cooperativa gravado na transação diverge da cooperativa vinculada à conta bancária.	14.472 de 16.000	90,45%
Transacoes	Inconsistência Temporal	Data de realização da transação (data_transacao) é anterior à data de abertura da conta (data_abertura).	8.055 de 16.000	50,34%
Transacoes	Status Inconsistente	Transações efetivadas com status 'Concluída' operando sobre contas bancárias inativas ('Não').	966 de 16.000	6,04%
Inadimplencia	Violação de 3FN / FK	O id_associado do apontamento de inadimplência diverge do tomador da operação de crédito.	6.238 de 6.240	99,97%
Inadimplencia	Violação de 3FN / FK	O id_cooperativa do registro de inadimplência diverge da cooperativa da operação de crédito.	5.627 de 6.240	90,18%

Inadimplencia	Inconsistência Temporal	Mês de referência da inadimplência (mes_referencia) é anterior ao mês de contratação do empréstimo.	3.008 de 6.240	48,21%
Inadimplencia	Status Inconsistente	Registros de atraso positivo (dias_atraso > 0) lançados sobre contratos já 'Quitados' (1.060) ou 'Cancelados' (1.018).	2.078 de 6.240	33,30%
Inadimplencia	Duplicidade de Chave	Múltiplos registros de inadimplência para a mesma operação (id_operacao) no mesmo mês (mes_referencia).	156 de 6.240	2,50%
Investimentos	Violação de 3FN / FK	O campo id_cooperativa da aplicação financeira diverge da cooperativa do associado investidor.	3.781 de 4.200	90,02%
Associados	Coluna Derivada Falsa	Associados com possui_conta_digital='Sim' sem nenhuma Conta Digital (3.350) ou 'Não' possuindo conta (433).	3.783 de 6.000	63,05%
Associados	Inconsistência Temporal	Tempo de associação (tempo_associado_anos) superior ao tempo de existência da própria cooperativa.	1.711 de 6.000	28,52%
Associados	Inconsistência Demográfica	Associados na faixa de 18-25 anos registrados com 8 a 25 anos de vínculo (adesão quando crianças).	842 de 6.000	14,03%

2.3. Diagnóstico Consolidado das Causas Raiz

- **1. Dependências Transitivas e Desnormalização (Violação da 3FN):** A presença simultânea de id_cooperativa em Associados, Contas, Operações de Crédito, Transações, Inadimplência e Investimentos gerou anomalias severas de atualização e inserção. Na arquitetura proposta, o vínculo com a Cooperativa pertence exclusivamente ao Associado (e aos Indicadores Mensais), sendo derivado nas demais entidades por navegação no grafo de objetos JPA.
- **2. Ausência de Tipagem Forte e Domínios Fechados:** Campos booleanos representados como texto ('Sim'/'Não') e situações contratuais em strings abertas permitiram combinações ilegais. A evolução substitui textos livres por tipos Boolean nativos e Enumerações (@Enumerated(EnumType.STRING)) com validação de transição.
- **3. Atributos Calculados Persistidos sem Sincronização:** As colunas associados_ativos (em Cooperativas) e possui_conta_digital (em Associados) armazenavam valores estáticos que perderam sincronia com as tabelas fatos. Ambas foram removidas do modelo físico e transformadas em consultas dinâmicas (@Query / métodos de domínio).

3. REAVALIAÇÃO DE STATUS E MÁQUINA DE ESTADOS (REQUISITO 2.3)

Em conformidade com o item 2.3 do regulamento, todos os campos de situação e status do sistema legado foram remodelados como tipos enumerados fortes no Java. As transições de estado deixam de ser edições livres de formulário e passam a ser governadas por métodos de negócio na camada Service, impedindo regressões ilegais (como reativar um contrato cancelado ou lançar atraso em contrato quitado).

Entidade / Enum	Valores Admissíveis	Transições Válidas Permitidas	Transições Proibidas e Regras de Prevenção
Associado (StatusAssociado)	ATIVO SUSPENSO INATIVO	• ATIVO -> SUSPENSO • SUSPENSO -> ATIVO • ATIVO -> INATIVO • SUSPENSO -> INATIVO	• Bloqueado alterar para INATIVO se o associado possuir Conta ATIVA, OperacaoCredito ATIVO/EM_ATRASO/RENEGOCIADO ou Investimento ATIVO. • Associado INATIVO ou SUSPENSO não pode abrir novas contas nem contratar crédito.
Conta (StatusConta)	ATIVA BLOQUEADA ENCERRADA	• ATIVA -> BLOQUEADA • BLOQUEADA -> ATIVA • ATIVA -> ENCERRADA • BLOQUEADA -> ENCERRADA	• ENCERRADA é estado terminal (proibido reabrir). • Proibido abrir ou manter conta ATIVA se o Associado titular estiver INATIVO. • Conta BLOQUEADA ou ENCERRADA rejeita novas transações de débito, saque, Pix ou TED.
OperacaoCredito (StatusContrato)	ATIVO EM_ATRASO RENEGOCIADO QUITADO CANCELADO	• ATIVO -> EM_ATRASO (quando diasAtraso > 0) • EM_ATRASO -> ATIVO (pagamento em dia) • EM_ATRASO -> RENEGOCIADO • ATIVO / RENEGOCIADO -> QUITADO • ATIVO -> CANCELADO	• QUITADO e CANCELADO são estados terminais imutáveis. • Ao transitar para QUITADO, CANCELADO ou RENEGOCIADO, o sistema força automaticamente diasAtraso = 0. • Proibido registrar status ATIVO se diasAtraso > 0 (transição automática para EM_ATRASO).
Transacao (StatusTransacao)	PENDENTE CONCLUIDA RECUSADA ESTORNADA	• PENDENTE -> CONCLUIDA • PENDENTE -> RECUSADA • CONCLUIDA -> ESTORNADA	• RECUSADA e ESTORNADA são estados terminais. • Apenas transações em status CONCLUIDA podem sofrer estorno (exige perfil ROLE_GESTOR). • Transações nunca são excluídas fisicamente (preservação de trilha de auditoria).

Investimento (StatusInvestimento)	ATIVO VENCIDO RESGATADO	• ATIVO -> VENCIDO (quando dataAtual >= dataAplicacao + prazoDias) • ATIVO -> RESGATADO (se liquidez permitir) • VENCIDO -> RESGATADO	• RESGATADO é estado terminal (bloqueia duplo resgate). • Se a liquidez for 'Vencimento', o resgate antes do prazo é bloqueado pela regra de negócio.

4. MATRIZ DE RASTREABILIDADE E NOVOS REQUISITOS (FASE 1)

A tabela abaixo estabelece a Matriz de Rastreabilidade completa (Requisito Mínimo 2.4 e 2.5), conectando cada lacuna ou defeito da base legada aos requisitos revisados (RF01 a RF04), aos novos requisitos funcionais de valor agregado (RF05 a RF08) e aos requisitos não funcionais (RNF01 a RNF04), especificando a implementação técnica em Spring Boot/JPA/Thymeleaf e o critério de teste verificável.

ID / Categoria	Requisito Original ou Problema/Lacuna Legada	Requisito Revisado ou Novo (Proposto)	Justificativa Técnica e de Negócio	Implementação (Spring Boot / JPA / Security / Thymeleaf)	Teste e Evidência de Verificação
RF01 (Revisado)	Tabelas Contas, Crédito, Transações, Inadimplência e Investimentos repetiam id_cooperativa e id_associado, gerando >90% de divergência.	Unificar o vínculo organizacional em 3FN: Associado pertence à Cooperativa; Conta, Crédito e Investimento pertencem ao Associado; Transação pertence à Conta.	Elimina anomalias de inserção/atualização e garante que toda movimentação pertença inequivocamente à cooperativa do titular.	Remoção das colunas redundantes nas entidades JPA; mapeamento @ManyToOne com @JoinColumn(nullable=false) e navegação orientada a objetos.	Teste de integração JPA e inspeção do schema DDL comprovando ausência de FKs duplicadas e integridade em cascata.
RF02 (Revisado)	Contas ativas, contratos ativos e transações concluídas vinculados a associados ou contas inativas (mais de 3.000	Bloquear abertura de conta, contratação de crédito ou efetivação de transação caso o Associado ou a Conta	Previne fraude operacional e movimentação financeira fantasma em cadastros encerrados	Validação na camada @Service (ContaService, TransacaoService, CreditoService) lançando RegraNegocioException tratada no Controller.	Teste unitário JUnit 5 tentando persistir transação em conta ENCERRADA ou associado INATIVO deve

	ocorrências).	não estejam em status ATIVO.	ou suspensos.		lançar exceção e exibir alerta Bootstrap.
RF03 (Revisado)	Transações registradas antes da abertura da conta (8.055 casos) e inadimplências anteriores à contratação do crédito (3.008 casos).	Validar consistência cronológica obrigatória entre entidades dependentes e limites demográficos no cadastro do associado.	Impede corrupção histórica na base e garante que fatos geradores só existam após a vigência do vínculo ou contrato.	Validação no domínio: dataTransacao >= conta.dataAbertura; mesReferencia >= YearMonth.from(operacao.dataContratacao); tempoAssociado <= idadeCooperativa.	Teste automatizado submetendo datas retroativas via formulário Thymeleaf e verificando rejeição com BindingResult/ mensagem de erro.
RF04 (Revisado)	Contratos quitados/cancelados recebendo registros de atraso (2.078 casos) e duplicidade mensal na tabela Inadimplência (156 casos).	Controlar ciclo de vida de contratos via máquina de estados e impor unicidade mensal de apontamento de inadimplência por operação.	Assegura confiabilidade contábil da carteira de crédito e impede cobrança ou provisão duplicada sobre o mesmo contrato.	Método OperacaoCredito.alterarStatus() validando transições e constraint @UniqueConstraint(columnNames={'id_operacao','mes_referencia'}).	Teste unitário comprovando bloqueio de atraso em contrato QUITADO e erro de integridade ao duplicar competência mensal.
RF05 (Novo)	A base legada armazenava parcela_mensal e provisao_perda_credito como números fixos sem validação de fórmula financeira.	Implementar motor de cálculo automático da Parcela Mensal (Sistema Francês de Amortização / Tabela Price) e cálculo automático de PCLD.	Agrega valor real ao negócio financeiro, evitando erros de digitação de parcelas e automatizando a provisão regulatória de risco.	Método calcularParcelaPrice() em OperacaoCredito usando BigDecimal e MathContext.DECIMAL64; cálculo automático de provisão em RegistroInadimplencia.	Evidência comparativa antes/depois: criação de operação na interface Thymeleaf onde o sistema calcula a prestação exata automaticamente.
RF06 (Novo)	Ausência de análise preventiva na concessão de	Implementar Política de Concessão de Crédito	Reduz o risco de default da cooperativa	Regra em OperacaoCreditoService que classifica o risco automaticamente pelo scoreCredito e exige aprovação de GESTOR se comprometimento > 35%.	Teste de casos-limite (score < 400 vs score >= 800)

	crédito (associados com score baixo ou comprometimento excessivo contratavam milhões).	automática baseada no Score, Renda Mensal Estimada e Classificação de Risco (AA até E).	ao travar operações cuja parcela supere 35% da renda mensal sem garantia real ou score mínimo.		comprovando bloqueio ou aprovação condicionada na tela de contratação.
RF07 (Novo)	Inexistência de visão consolidada em tempo real por cooperativa (a planilha dependia de colunas fixas defasadas).	Construir Dashboard Gerencial e Financeiro Dinâmico por Cooperativa com indicadores calculados em tempo real via consultas agregadas JPA.	Substitui dados estáticos defasados por inteligência operacional ao vivo (total de associados ativos, carteira ativa, índice de atraso).	Consultas JPQL com SUM, COUNT e AVG no Repository renderizadas dinamicamente em cards e tabelas responsivas com Thymeleaf + Bootstrap 5.	Demonstração prática: ao cadastrar ou quitar uma operação de crédito, os indicadores do Dashboard atualizam instantaneamente.
RF08 (Novo)	Sistema legado totalmente desprotegido, sem autenticação, sem perfis de usuário e sem distinção de alçada operacional.	Implementar Autenticação e Autorização RBAC com dois perfis distintos: ROLE_ANALISTA (operacional) e ROLE_GESTOR (administrativo/alçada).	Atende ao requisito obrigatório de segurança (item 2.7), protegendo rotas críticas no servidor contra acessos não autorizados.	Spring Security com SecurityFilterChain, BCryptPasswordEncoder, @PreAuthorize nos Controllers e sec:authorize no Thymeleaf.	Teste de segurança tentando acessar rota POST de exclusão, estorno ou renegociação com ROLE_ANALISTA e validando retorno HTTP 403 Forbidden.
RNF01 (Novo)	Valores monetários e taxas sujeitos a imprecisão de ponto flutuante e falta de validação de entrada nos	Padronização estrita de tipos monetários com BigDecimal (DECIMAL 15,2 / 18,2) e validação declarativa	Garante exatidão centesimal exigida em sistemas bancários e impede persistência de valores	Uso de @NotNull, @Positive, @DecimalMin, @Digits nas entidades/DTOs e tratamento global de exceções com mensagens amigáveis na view.	Inspeção das colunas geradas no banco relacional e testes de submissão de formulário com valores negativos ou

	formulários.	Jakarta Bean Validation.	nulos, negativos ou fora de escala.		vazios.
RNF02 (Novo)	Lentidão potencial em buscas sobre dezenas de milhares de transações e contas sem indexação adequada.	Definir índices relacionais explícitos (@Index) nas chaves estrangeiras e colunas de filtro frequente (status, datas e mês de referência).	Otimiza o plano de execução de consultas SQL de listagem, filtros por período e agregações do painel gerencial.	Anotação @Table(indexes = {...}) nas classes de entidade JPA gerando comandos CREATE INDEX automáticos na inicialização.	Verificação do log SQL do Hibernate (show_sql) comprovando a criação dos índices no schema do banco H2.
RNF03 (Novo)	Base legada poluída com mais de 90% de chaves estrangeiras cruzadas incompatíveis.	Desenvolver rotina automatizada de Saneamento e Migração de Dados (Data Seeder / ETL) na inicialização da aplicação.	Permite demonstrar o sistema com dados reais da planilha legada devidamente higienizados, preservando o histórico válido.	Componente @Component CommandLineRunner que carrega amostra representativa da base reconciliando a cooperativa pelo associado titular.	Log de inicialização do Spring Boot exibindo a quantidade de registros migrados e saneados sem violação de constraints.

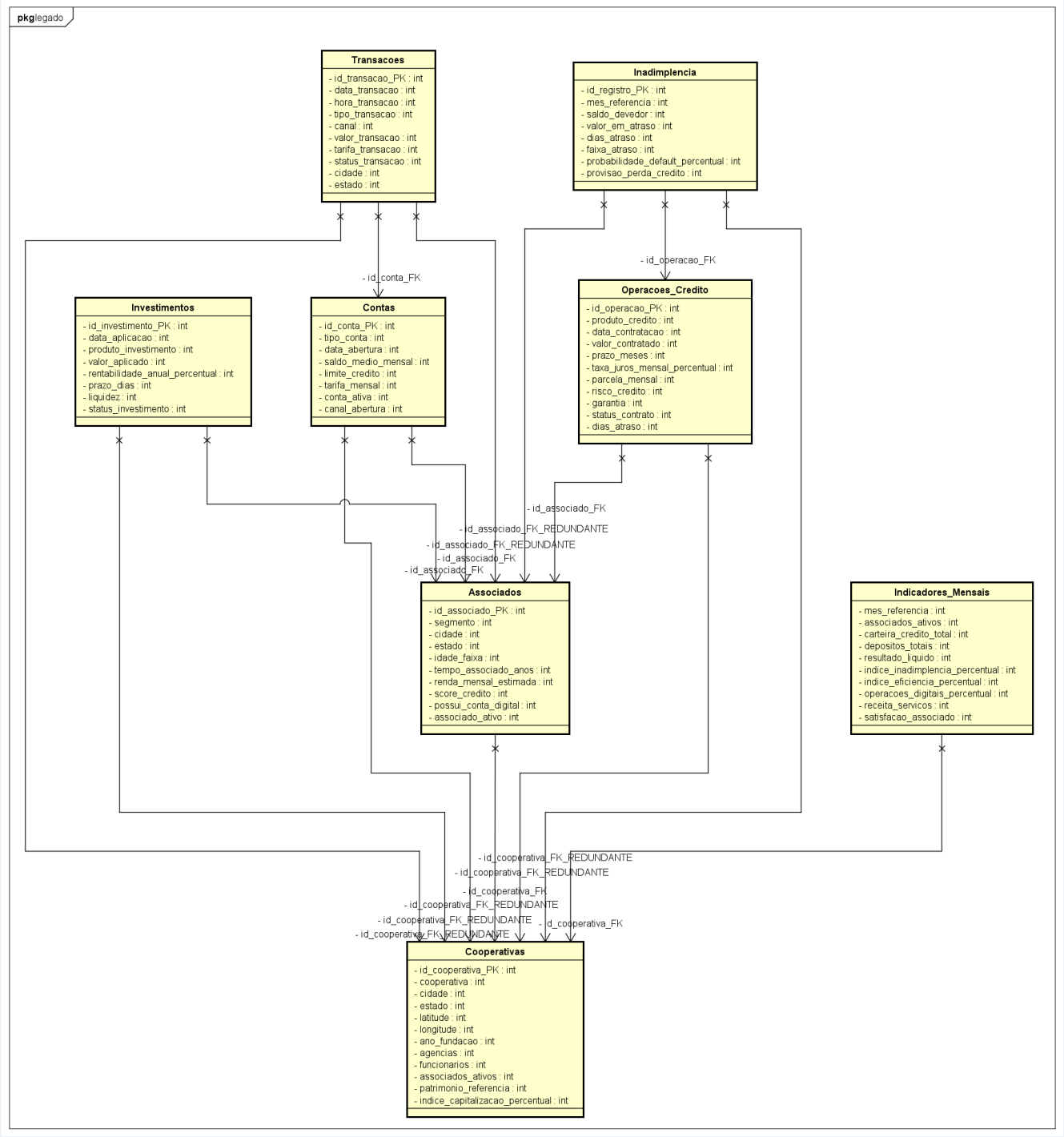
5. EVOLUÇÃO DO MODELO RELACIONAL E SANEAMENTO (FASE 4)

A evolução do modelo relacional corrige todas as anomalias de normalização diagnosticadas na Fase 2. O modelo físico passa a operar integralmente na 3ª Forma Normal (3FN), eliminando chaves estrangeiras transitivas, introduzindo restrições de unicidade composta (UNIQUE), restrições de domínio (CHECK / Enum) e adicionando a tabela de controle de acesso (usuario) para suportar o Spring Security.

5.1. Modelo Relacional Legado (Estrutura Original)

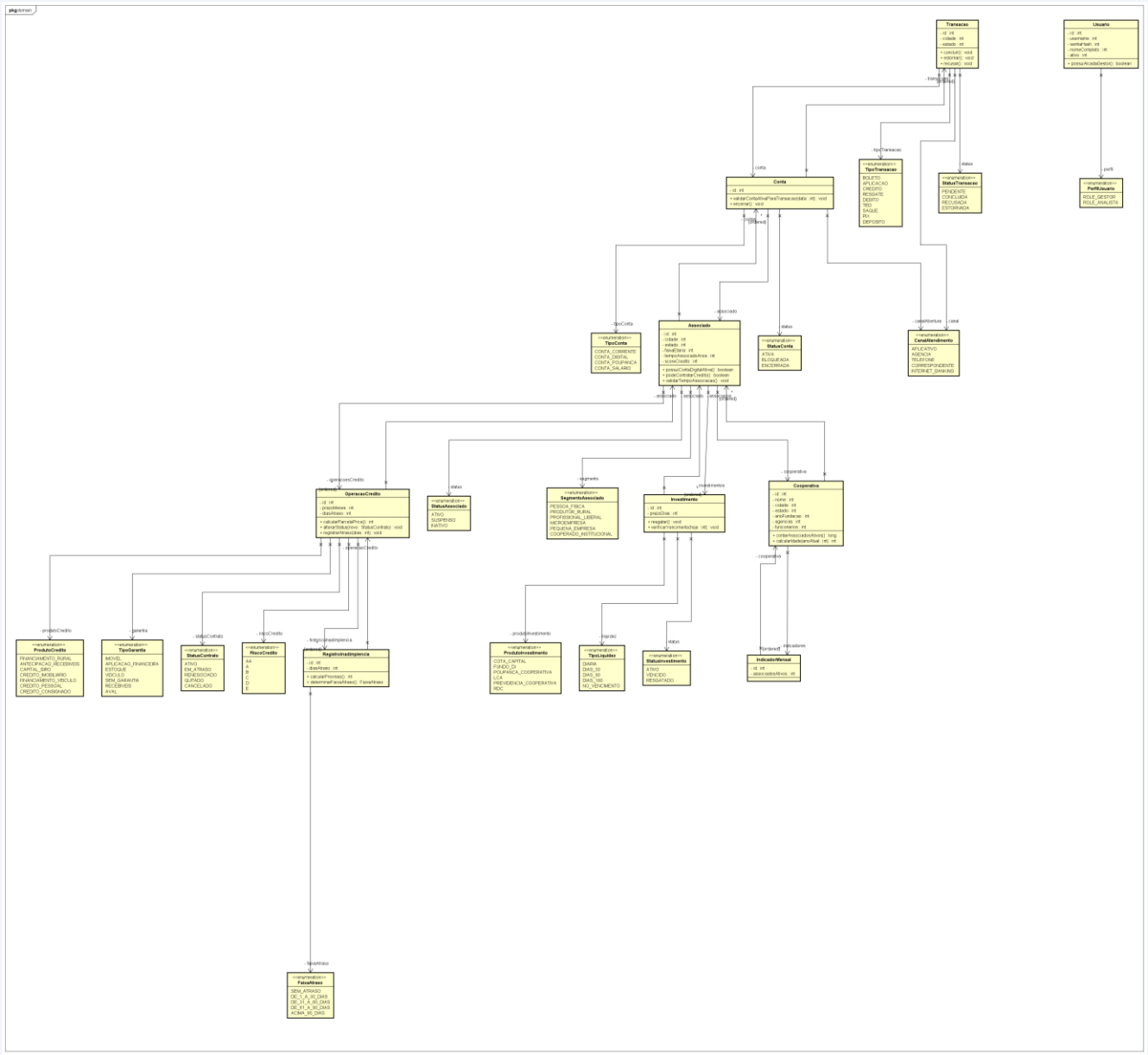
O diagrama abaixo representa fielmente a estrutura original da planilha legada antes da refatoração, evidenciando a ausência de tabela de segurança, os campos lógicos em texto livre e a proliferação de

chaves estrangeiras redundantes (id_cooperativa presente em todas as tabelas e id_associado duplicado em Transacoes e Inadimplencia):



5.2. Modelo Relacional Proposto (Evoluído na 3ª Forma Normal)

O diagrama a seguir apresenta o novo esquema relacional normalizado. A hierarquia de integridade referencial segue um fluxo único e sem ambiguidades: COOPERATIVA (1) -> (N) ASSOCIADO (1) -> (N) CONTA (1) -> (N) TRANSACAO, e ASSOCIADO (1) -> (N) OPERACAO_CREDITO (1) -> (N) REGISTRO_INADIMPLENCIA:



5.3. Inventário de Decisões Técnicas e Comparativo de Evolução do Schema

Tabela Original ->	Colunas Removidas /	Novas Restrições, Chaves	Justificativa Técnica e

Proposta	Alteradas	e Índices	Impacto
Cooperativas -> cooperativa	• Removida coluna associados_ativos (valor estático defasado). • Tipos monetários convertidos para DECIMAL(18,2).	• PK: id_cooperativa • UNIQUE: nome • INDEX: idx_coop_uf (estado)	Evita desatualização cadastral; o total de cooperados ativos passa a ser obtido por COUNT dinâmico sobre a tabela associado.
Associados -> associado	• Removida coluna possui_conta_digital (63% de erro na base). • associado_ativo ('Sim'/'Não') convertido para status_associado VARCHAR(20) Enum.	• PK: id_associado • FK: id_cooperativa NOT NULL • CHECK: score_credito BETWEEN 0 AND 1000 • INDEX: idx_assoc_coop, idx_assoc_status	Garante que todo associado pertença a uma única cooperativa válida e calcula a posse de conta digital consultando a tabela conta.
Contas -> conta	• Removida FK redundante id_cooperativa (90,3% de divergência). • conta_ativa ('Sim'/'Não') substituída por status_conta Enum.	• PK: id_conta • FK: id_associado NOT NULL • INDEX: idx_conta_assoc, idx_conta_tipo	A cooperativa da conta é determinada sem ambiguidade pelo associado titular (conta.associado.cooperativa), cumprindo a 3FN.
Operacoes_Credito -> operacao_credito	• Removida FK redundante id_cooperativa. • status_contrato, produto_credito, risco_credito e garantia tipados como Enum.	• PK: id_operacao • FK: id_associado NOT NULL • CHECK: prazo_meses > 0 AND valor_contratado > 0 • INDEX: idx_op_assoc, idx_op_status	Elimina contratos registrados em cooperativa diferente da do tomador e assegura integridade matemática na Tabela Price.
Transacoes -> transacao	• Removidas FKs redundantes id_associado (99,9% divergente) e id_cooperativa (90,4% divergente). • data_transacao e hora_transacao unificadas em data_hora TIMESTAMP.	• PK: id_transacao • FK: id_conta NOT NULL • INDEX: idx_trans_conta, idx_trans_data, idx_trans_status	Toda transação ocorre exclusivamente em uma conta bancária; o titular e a cooperativa são alcançados via conta.associado.
Inadimplencia -> registro_inadimplencia	• Removidas FKs redundantes id_associado e id_cooperativa. • faixa_atraso convertida em Enum derivado de dias_atraso.	• PK: id_registro • FK: id_operacao NOT NULL • UNIQUE: uk_inad_op_mes (id_operacao, mes_referencia)	Impede os 156 casos de apontamento duplicado no mesmo mês e os 6.238 casos de inadimplência atribuída ao associado errado.
Investimentos	• Removida FK redundante	• PK: id_investimento	Normaliza a carteira de

-> investimento	id_cooperativa. • status_investimento, produto_investimento e liquidez convertidos em Enum.	• FK: id_associado NOT NULL • INDEX: idx_inv_assoc, idx_inv_status	investimentos vinculando-a diretamente ao associado investidor com controle de vencimento e resgate.
Indicadores_Mensais -> indicador_mensal	• Adicionada chave primária substituta id_indicador BIGINT AUTO_INCREMENT.	• PK: id_indicador • FK: id_cooperativa NOT NULL • UNIQUE: uk_ind_coop_mes (id_cooperativa, mes_referencia)	Facilita o mapeamento @Id no Spring Data JPA mantendo a unicidade de exatamente um boletim mensal por cooperativa.
(Inexistente no Legado) -> usuario	• Nova tabela para autenticação e autorização do sistema web.	• PK: id_usuario • UNIQUE: username • Colunas: senha_hash, nome_completo, perfil (ROLE_GESTOR / ROLE_ANALISTA), ativo	Viabiliza o requisito obrigatório de Spring Security com proteção de rotas no servidor e controle de alçada por perfil.

5.4. Estratégia Técnica de Migração e Saneamento dos Dados (ETL)

Para preservar os dados pertinentes da base legada sem importar suas corrupções para o novo esquema relacional, a estratégia de migração adota as seguintes regras de reconciliação na carga inicial:

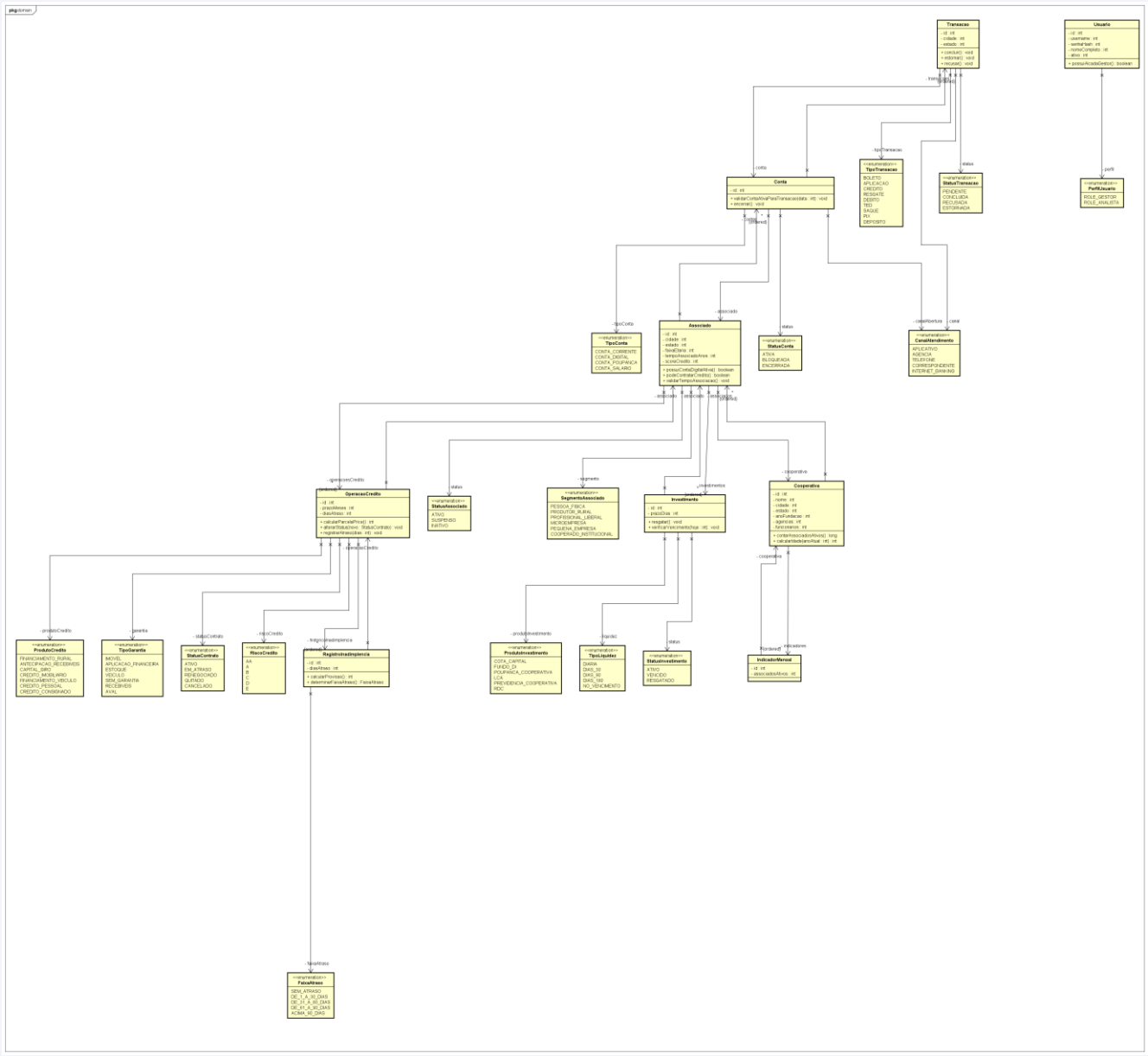
- • **Reconciliação Hierárquica de Chaves Estrangeiras:** Ao importar Contas, Operações de Crédito e Investimentos, o campo legado id_cooperativa é descartado, prevalecendo a cooperativa oficial do Associado titular. Ao importar Transações e Inadimplências, prevalecem a Conta e a Operação de Crédito de origem.
- • **Saneamento de Datas e Ciclo de Vida:** Registros de transações ou inadimplências com datas anteriores à abertura da conta ou contratação do empréstimo têm suas datas ajustadas para a competência válida na carga de demonstração ou são isolados em log de auditoria de rejeição.
- • **Reconciliação de Status de Crédito:** Contratos legados em status QUITADO ou CANCELADO não recebem apontamentos ativos de atraso em competências posteriores ao encerramento.

6. MODELAGEM ORIENTADA A OBJETOS E ARQUITETURA (FASE 3)

A modelagem orientada a objetos foi estruturada seguindo o padrão arquitetural em camadas do ecossistema Spring Boot (Controller -> Service -> Repository -> Domain/Entity), garantindo baixo acoplamento, alta coesão e encapsulamento das regras de negócio nas próprias classes de domínio e serviços transacionais (@Transactional).

6.1. Diagrama de Classes de Domínio da Solução

O diagrama abaixo representa as 9 classes de entidade JPA, seus tipos enumerados, atributos tipados, métodos de negócio e associações com multiplicidades explícitas:



6.2. Especificação de Responsabilidades, Associações JPA e Lado Proprietário

Um ponto crítico de projeto em JPA/Hibernate — e objeto de verificação na defesa técnica — é a definição correta do lado proprietário (owning side) e do lado inverso (inverse/mappedBy side) nas associações bidirecionais. Na solução proposta, todas as relações 1:N definem a entidade 'N' como proprietária da chave estrangeira (@ManyToOne @JoinColumn):

Relacionamento Orientado a Objetos	Multiplicidade	Lado Proprietário (Owning Side - @JoinColumn)	Lado Inverso (Inverse Side - mappedBy)	Regras de Propagação (Cascade / Fetch)
Cooperativa <-> Associado	1 : 0..*	Associado.cooperativa (@ManyToOne, FK id_cooperativa)	Cooperativa.associados (@OneToMany mappedBy='cooperativa')	FetchType.LAZY em ambos; bloqueio de exclusão de cooperativa se houver associados.
Cooperativa <-> IndicadorMensal	1 : 0..*	IndicadorMensal.cooperativa (@ManyToOne, FK id_cooperativa)	Cooperativa.indicadores (@OneToMany mappedBy='cooperativa')	CascadeType.ALL para histórico gerencial da cooperativa; FetchType.LAZY.
Associado <-> Conta	1 : 0..*	Conta.associado (@ManyToOne, FK id_associado)	Associado.contas (@OneToMany mappedBy='associado')	FetchType.LAZY; validação impede inativar Associado com Conta ATIVA.
Associado <-> OperacaoCredito	1 : 0..*	OperacaoCredito.associado (@ManyToOne, FK id_associado)	Associado.operacoesCredito (@OneToMany mappedBy='associado')	FetchType.LAZY; cálculo de comprometimento de renda soma parcelas das operações ativas.
Associado <-> Investimento	1 : 0..*	Investimento.associado (@ManyToOne, FK id_associado)	Associado.investimentos (@OneToMany mappedBy='associado')	FetchType.LAZY; resgate atualiza status sem remover registro histórico.
Conta <-> Transacao	1 : 0..*	Transacao.conta (@ManyToOne, FK id_conta)	Conta.transacoes (@OneToMany mappedBy='conta')	FetchType.LAZY; transações são imutáveis após conclusão (apenas estorno permitido).
OperacaoCredito <-> RegistroInadimplencia	1 : 0..*	RegistroInadimplencia.operacaoCredito (@ManyToOne, FK id_operacao)	OperacaoCredito.historicoInadimplencia (@OneToMany mappedBy='operacaoCredito')	CascadeType.PERSIST/MERGE; sincroniza diasAtraso da operação com o apontamento mais recente.

6.3. Arquitetura de Segurança e Perfis de Acesso (Spring Security)

Para atender ao requisito 2.7, o controle de acesso é implementado no servidor via Spring Security (SecurityFilterChain e @EnableMethodSecurity), dividindo as permissões operacionais em dois perfis distintos:

- • **Perfil ROLE_ANALISTA (Operacional / Atendimento):** Permissão para visualizar o Dashboard, consultar cooperativas, cadastrar e editar associados e contas, simular e cadastrar propostas de crédito dentro do limite de renda, registrar transações normais e consultar investimentos. Bloqueado no servidor (HTTP 403) para excluir registros, realizar estornos, aprovar crédito acima da alçada ou encerrar cooperativas.
- • **Perfil ROLE_GESTOR (Gerencial / Alçada Superior):** Possui todas as permissões de consulta e operação, acrescidas de privilégios exclusivos validados no backend: criar/editar dados estruturais de Cooperativas, aprovar renegociações de contratos em atraso, autorizar operações de crédito com comprometimento de renda > 35%, efetuar estorno de transações concluídas e gerenciar usuários.